

BBT RADAR — Failure Prediction, Environment Compare & Dev-PR Correlation

Architecture Design Document — **MD-Knowledge-Base Variant (No LLM)**

> **Status:** Draft v1.0 (sibling variant of PREDICTION-AND-PR-CORRELATION.md)

> **Scope:** Same three capabilities (R1 Prediction column, R2 Two-environment Compare, R3 Dev-PR Correlation), but the prediction/explanation engine is built **entirely from a curated Markdown knowledge base** — no LLM, no cloud-AI dependency.

> **Base codebase:** `WK.Axcess.ReturnDashboard` — React 18 + Vite client, Node 18 + Express + TypeScript server, Azure DevOps integration via PAT.

1. Why This Variant Exists

The primary architecture relies on AWS Bedrock for the "long tail" of unknown failures. This variant removes the LLM dependency and replaces it with a **versioned Markdown Knowledge Base (MD-KB)** authored and maintained by SDETs. Use this variant when any of the following apply:

- No approved cloud-AI vendor (AWS / Azure OpenAI / Mistral) yet.
- Strict data-residency rules forbid sending failure messages off-prem.
- Zero per-call cost is a hard requirement.
- The team prefers fully-deterministic, auditable, code-reviewable predictions.

The trade-off: novel failures that don't yet have an MD-KB entry are rendered as `Unknown — please add to KB`, with a one-click **"Suggest KB entry"** flow that pre-fills a draft `.md` file and opens a PR against the KB repo.

2. Executive Summary

Same three capabilities as the LLM variant; same UI surfaces; same export formats. The differences are limited to the **prediction engine** and **feedback loop**:

| # | Capability | UI Surface | Core Output |

| --|-----|-----|-----|

| **R1** | **Failure Prediction** | on Analyze Build | New **Prediction** column | Per-failure category + reason rendered from the matching MD-KB entry |

| **R2** | **Environment Compare** | on Compare Builds | **Env 1 vs Env 2** mode | Deterministic set-math diff + a **template-rendered narrative** built from per-failure KB entries |

| **R3** | **Dev-PR Correlation** | **Analyze with PR** input | Per failed test: `likelyCausedByPR`, confidence, explanation **computed from rules + KB**, suspect hunks |

Engine substitution at a glance:

| Concern | LLM variant | **This variant (MD-KB)** |

| ---|---|---|

| Long-tail prediction | AWS Bedrock Claude 3.5 | **MD-KB lookup** (fuzzy match on error signature) |

| Narrative for env diff | LLM summarization | **Template engine** (Handlebars-style) fed by KB entries + bucket counts |

| Few-shot exemplars | RAG vector store (Titan embeddings) | **MD-KB itself is the "memory"** — every approved fix is a new `.md` entry |

| User feedback | Updates RAG + cache | **Opens a PR** against the KB repo (`kb-pr` action) and updates a local override JSON |

| Cost / latency | \$ per call, ~1–3 s | \$0, <50 ms (in-memory match) |

| Cloud dependency | AWS | **None** |

3. Design Decisions (Variant Outcomes)

| Q | Decision | Rationale |

|---|-----|-----|

| **Q1** — Prediction engine | **Rules + MD-KB lookup** | Deterministic, explainable, zero external cost, version-controlled by SDETs.

|

| **Q2** — LLM provider | **None** | Replaced by MD-KB + fuzzy matcher (Fuse.js / minisearch). |

| **Q3** — Environment model | **Hybrid: registry + free-form override** (unchanged) | Same as primary variant. |

| **Q4** — Diff semantics | **Set-math diff + template-rendered narrative** | Narrative composed by Handlebars template from KB entries; no LLM. |

| **Q5** — PR source | **Both ADO Repos + GitHub, auto-detect** (unchanged) | Same as primary variant. |

| **Q6** — PR correlation | **File overlap + symbol/line proximity + KB rules** | LLM root-cause replaced by KB rule rendering against suspect hunks. |

| **Q7** — Persistence | **Flat JSON files + Markdown KB folder** | Predictions cache as JSON; knowledge as `.md` files under version control. |

| **Q8** — Feedback loop | **Approve / Reject / Edit \u2192 KB PR** | Edits draft a new `.md` and open a PR (`gh pr create` or ADO PR API). |

4. The Markdown Knowledge Base (MD-KB)

The KB is a **folder of `.md` files**, each describing one **failure pattern**. It can live in the same repo (`dashboard/kb/`) or a separate repo (`bbt-failure-kb`) mounted/cloned at startup. The server loads it into memory at boot and watches for changes.

4.1 Folder layout

...

dashboard/

└─ kb/

```
├─ README.md           (authoring guide)
├─ schema.json         (front-matter JSON Schema)
├─ _index.json         (build artifact: id → file map; rebuilt on boot)
├─ timeout/
│   ├─ selenium-element-wait.md
│   ├─ api-http-timeout.md
│   └─ pageload-timeout.md
├─ element-not-found/
│   ├─ stale-element-reference.md
│   └─ selector-changed.md
│   └─ iframe-context.md
├─ assertion-mismatch/
│   ├─ decimal-rounding.md
│   └─ locale-date-format.md
│   └─ tax-calc-edition-mismatch.md
├─ network-error/
│   └─ ado-feed-5xx.md
│   └─ azure-storage-throttled.md
└─ environment-down/
```

```

|   └─ qa-tenant-unreachable.md
└─ flaky/
|   └─ retry-passed.md
└─ data-dependency/
|   └─ prior-year-return-missing.md
|   └─ seed-data-drift.md
└─ pr-related/
|   └─ shared-util-refactor.md
|   └─ selector-rename.md
└─ env-diff/          (R2-specific narrative fragments)
    └─ qa-only-cluster.md
    └─ stage-only-cluster.md
    └─ shared-failure-cluster.md
...

```

4.2 KB entry file format

Each `.md` file has a YAML front-matter block (machine-readable) followed by Markdown body (human-readable, surfaced in the UI):

```

```markdown

id: kb.timeout.selenium-element-wait
category: Timeout
confidence: 0.92
match:
 errorMessageAny:
 - "TimeoutException"
 - "wait timed out"
 - "element click intercepted"
 durationTrend: [increasing, stable]
 minDurationMs: 30000
 sourceFileGlob: ["**/*UiTests/**", "**/*Selenium*"]
 reasonTemplate: |
 Selenium wait timed out after {{durationInMs}}ms on `{{methodName}}`.
 The element may be slow to render, hidden behind a modal, or the selector
 may be outdated.
 suggestedFixes:
 - Increase the wait to 60 s if the page is known to be slow.
 - Re-record the selector with the Selenium IDE; verify it survives DOM changes.
 - Add an explicit `waitForElementToBeClickable` before the action.
 relatedKbIds:
 - kb.element-not-found.stale-element-reference
owners:
 - sdet-platform@wolterskluwer.com
lastReviewed: 2026-05-01

When this happens

```

Most commonly seen on the **TPW Return Summary** page after a long recalculation. The wait is short because the original test was written for a smaller return.

## ## How to verify

1. Open the build in ADO and inspect the screenshot artifact for the failed step.
2. Re-run the single test locally with `dotnet test --filter``. If it reliably reproduces, this KB entry applies.

## ## Known good fix

See PR #1234 (`TaxWinAppAutomation``) for an example wait-time bump pattern.

---

### ### 4.3 Match grammar (front-matter `match:` block)

| Key | Type | Semantics |

|---|---|---|

| ``errorMessageAny`` | `string[]` | Case-insensitive substring OR — at least one must match ``failedTest.errorMessage``. |

| ``errorMessageAll`` | `string[]` | All must match. |

| ``errorMessageRegex`` | `string` | JS regex (string form), tried after the cheaper substring checks. |

| ``durationTrend`` | `enum[]` | Any of ``increasing`` / ``decreasing`` / ``stable`` from existing ``FailedTest``. |

| ``minDurationMs`` / ``maxDurationMs`` | `number` | Bounds on ``durationInMs``. |

| ``sourceFileGlob`` | `string[]` | Minimatch globs against ``failedTest.sourceFile``. |

| ``methodNameRegex`` | `string` | Optional regex against ``failedTest.methodName``. |

| ``runNameAny`` | `string[]` | Match by ADO test run name (useful for ``EnvironmentDown`` patterns). |

| ``aggregate`` | `object` | Cross-test conditions, evaluated against the *whole* build (e.g., ``"minFailedFraction": 0.5`` for ``EnvironmentDown``).

|

The matcher computes a **score** (count of satisfied keys, weighted) and picks the highest-scoring entry whose score × ``confidence`` ≥ threshold (default ``0.6``). Ties are broken by ``lastReviewed`` (newer wins).

### ### 4.4 KB authoring guide (`kb/README.md``)

- One ``md`` per pattern; file name is kebab-case of the pattern.
- Front-matter required; body optional but recommended (drives the "Why / How to verify / Known fix" UI panel).
- Run ``npm run kb:validate`` (new script) — validates every file against ``schema.json`` and prints conflicts (e.g., two entries that would match the same example).
- PR-review checklist embedded in ``README.md``.

### ### 4.5 Server-side loader

``KbLoader.ts`` (new):

- Boot: read every ``md`` under ``dashboard/kb/**/*md``, parse front-matter via ``gray-matter``, validate via ``ajv``, build ``_index.json``.
- Watch: ``chokidar`` on the ``kb/`` folder; hot-reload on file change (no server restart). Same pattern as ``patStore.ts``.
- Expose: ``KbLoader.matchOne(failedTest, aggregateContext?)`` → ``KbMatch | null``.

---

## ## 5. Architecture (delta vs primary variant)

SERVER (Express)		
routes/		
analyze.ts	(extended — calls PredictionService)	
predict.ts	(NEW)	
envCompare.ts	(NEW)	
prCorrelate.ts	(NEW)	
feedback.ts	(NEW — opens KB PRs)	
kb.ts	(NEW — list, search, reload, validate, suggest)	
services/		
PredictionService.ts	(NEW — orchestrator: rules → KB → cache)	
RuleClassifier.ts	(NEW — same as primary variant)	
KbLoader.ts	(NEW — load + watch + match MD files)	← replaces BedrockClient
KbMatcher.ts	(NEW — scoring + tie-break)	← replaces RAG retrieval
NarrativeRenderer.ts	(NEW — Handlebars + KB fragments)	← replaces LLM narrative
EnvCompareService.ts	(NEW — set-math + NarrativeRenderer)	
PrCorrelationService.ts	(NEW — overlap + proximity + KB rules)	
FeedbackStore.ts	(NEW — append + KB PR draft)	
KbPrDrafter.ts	(NEW — turns user edits into a .md PR)	← replaces RagMemoryService
EnvironmentRegistry.ts	(NEW — unchanged from primary variant)	
providers/		
PrProvider.ts, AdoPrProvider.ts, GitHubPrProvider.ts		
existing (unchanged usage):		
adoClient.ts, analyzer.ts, bugAnalyser.ts, patStore.ts		

Client-side components are **identical** to the primary variant (`AnalyzeBuildView`, `CompareBuildsView`, `PredictionCell`, `FeedbackButtons`, `PrInputPanel`, `EnvSelector`, `DiffTable`, `DiffNarrative`), except:`

- — —

## ...

```

dashboard/
├── kb/ (NEW — Markdown knowledge base)
│ ├── README.md
│ ├── schema.json
│ ├── _index.json (generated)
│ └── <category>/<pattern>.md (many)

```

```

└─ src/
 └─ client/
 └─ analyze/ (NEW directory — same as primary)
 └─ AnalyzeBuildView.tsx
 └─ CompareBuildsView.tsx
 └─ PredictionCell.tsx (adds 📖 KB link)
 └─ FeedbackButtons.tsx (Edit → KB Draft modal)
 └─ KbDraftModal.tsx (NEW — variant-only)
 └─ KbDrawer.tsx (NEW — variant-only; renders .md body)
 └─ PrInputPanel.tsx
 └─ EnvSelector.tsx
 └─ DiffTable.tsx
 └─ DiffNarrative.tsx
 └─ api/
 └─ predictionApi.ts
 └─ envCompareApi.ts
 └─ prCorrelateApi.ts
 └─ feedbackApi.ts
 └─ kbApi.ts (NEW — variant-only)
 └─ types/
 └─ prediction.ts
 └─ envCompare.ts
 └─ prCorrelation.ts
 └─ kb.ts (NEW — variant-only)
 └─ (existing files unchanged)
 └─ server/
 └─ routes/
 └─ analyze.ts (extended)
 └─ predict.ts (NEW)
 └─ envCompare.ts (NEW)
 └─ prCorrelate.ts (NEW)
 └─ feedback.ts (NEW)
 └─ kb.ts (NEW — variant-only)
 └─ services/
 └─ PredictionService.ts
 └─ RuleClassifier.ts
 └─ KbLoader.ts (NEW)
 └─ KbMatcher.ts (NEW)
 └─ NarrativeRenderer.ts (NEW)
 └─ EnvCompareService.ts
 └─ PrCorrelationService.ts
 └─ FeedbackStore.ts
 └─ KbPrDrafter.ts (NEW)
 └─ EnvironmentRegistry.ts
 └─ providers/
 └─ PrProvider.ts

```

```

| | | |— AdoPrProvider.ts
| | | |— GitHubPrProvider.ts
| | | |— (existing files unchanged)
| | |
| | |— data/
| | |— environments.json
| | |— predictions-cache.json
| | |— env-compare-cache.json
| | |— pr-correlation-cache.json
| | |— prediction-feedback.json (audit log)
| | |— kb-overrides.json (NEW — local edits awaiting PR merge)
| | |— kb-pr-queue.json (NEW — pending KB PR drafts)
| |
| |— docs/
| |— architecture/
| |— PREDICTION-AND-PR-CORRELATION.md (primary, LLM variant)
| |— PREDICTION-AND-PR-CORRELATION-MD-KB.md (this document)

```

---

---

## ## 7. Domain Model Extensions

All types from the primary variant apply. Additions / changes:

```ts

```

// FailurePrediction gets KB provenance
export interface FailurePrediction {
  category: 'Timeout' | 'ElementNotFound' | 'AssertionMismatch' | 'NetworkError'
    | 'EnvironmentDown' | 'Flaky' | 'DataDependency' | 'PrRelated' | 'Unknown';
  reason: string;          // rendered from KB entry reasonTemplate
  confidence: number;      // 0..1
  source: 'rule' | 'kb' | 'cache' | 'override' | 'unknown';
  ruleId?: string;
  kbId?: string;           // when source = 'kb'
  kbFilePath?: string;     // e.g., kb/timeout/selenium-element-wait.md
  matchedKeys?: string[];  // which match keys fired
  suggestedFixes?: string[]; // copied from KB entry
  bodyMarkdown?: string;   // streamed lazily to client when drawer opens
  predictedAt: string;
}

```

// KB types

```

export interface KbEntry {
  id: string;
  filePath: string;
  category: FailurePrediction['category'];
  confidence: number;
  match: KbMatchSpec;
  reasonTemplate: string;
}

```

```

suggestedFixes?: string[];
relatedKbIds?: string[];
owners?: string[];
lastReviewed?: string;
bodyMarkdown: string;      // raw body (after front-matter)
}

export interface KbMatch {
  entry: KbEntry;
  score: number;           // computed match score
  matchedKeys: string[];
}

export interface KbSuggestion {
  draftId: string;
  proposedFilePath: string; // e.g., kb/unknown/2026-05-24-flaky-summary-page.md
  frontMatter: Record<string, unknown>;
  body: string;
  basedOnFailedTest: FailedTest;
}
...

```

The `EnvCompareResult.narrative`` keeps the same shape; the strings inside are now produced by `NarrativeRenderer`` from KB fragments and bucket stats instead of an LLM.

8. API Endpoints

All endpoints from the primary variant remain; LLM-specific ones disappear and four KB endpoints are added.

8.1 Prediction (unchanged signatures)

| Method | Path | Body | Notes |
|--------|------------------------------|---|--|
| POST | /api/predict/failures`       | { buildUrl, pat?, useCache?: true, forceRefresh?: false } | forceLLM` replaced by forceRefresh` (re-run KB matcher). |
| POST | /api/predict/failure` | { failedTest, contextBuildId?, forceRefresh?: false } | Same. |
| GET | /api/predict/cache/:buildId` | — | Same. |
| DELETE | /api/predict/cache/:buildId` | — | Same. |

8.2 Environment Compare (unchanged)

Same three endpoints (`GET /api/environments``, `POST /api/env-compare``, `GET /api/env-compare/cache/:key``). `withNarrative`` now flips the `template renderer`` on/off instead of the LLM.

8.3 PR Correlation (unchanged signatures)

Same three endpoints. LLM root-cause step is replaced by `KbMatcher`` re-run against the failed test enriched with `suspectHunks`` (pr-related/*` KB entries are scoped to that category).

8.4 Feedback (semantics shift)

| Method | Path | Body | Response | Purpose |
|--------|----------------|---------|----------|----------------------|
| POST | /api/feedback` | { ... } | { ... } | Record user feedback |

| `POST` | `/api/feedback/prediction` | `{ predictionKey, action: 'approve' \| 'reject' \| 'edit' \| 'kb-pr', editedReason?, editedCategory?, suggestedFixes?, userId }` | `{ stored: true, overrideApplied: boolean, kbPrDraftId?: string }` | `approve` → write to `kb-overrides.json` so the same `(test, errSig)` keeps the chosen entry. `edit` → also queue a KB PR draft. `kb-pr` → create the actual PR via `KbPrDrafter`. |
| `GET` | `/api/feedback/stats` | `{ since=ISO` | `{ total, approved, rejected, edited, kbPrsOpened, kbCoverage, accuracyByCategory }` |
Dashboard. |

8.5 KB management (NEW — variant-only)

| Method | Path | Body | Response | Purpose |
|--------|--------------------|--|--------------------------------|--|
| `GET` | `/api/kb` | `{ category=&q=` | `{ entries: KbEntry[] }` | List + search the KB (excludes `bodyMarkdown` by default). |
| `GET` | `/api/kb/:id` | — | `KbEntry` (with body) | Fetch one entry for the side drawer. |
| `POST` | `/api/kb/reload` | — | `{ count, validated, errors }` | Force re-read of the `kb/` folder. |
| `POST` | `/api/kb/validate` | `{ frontMatter, body }` | `{ valid, errors? }` | Lints a draft before PR. |
| `POST` | `/api/kb/suggest` | `{ failedTest, draftReason, draftCategory, suggestedFixes }` | `KbSuggestion` | Builds a draft `.md` (does not commit). |
| `POST` | `/api/kb/pr` | `{ draftId }` | `{ prUrl, prId }` | Opens an actual PR against the KB repo via `KbPrDrafter`. |

8.6 Existing (unchanged)

`GET /api/health`, `GET /api/pat/status`, `POST /api/pat/reload`, `POST /api/analyze` (extended with `?withPrediction=true`).

9. Data Flows

9.1 R1 — Analyze Build with Prediction column (MD-KB)

...

User clicks "Analyze"

|

Client: POST /api/analyze?withPrediction=true

|

Server: analyze.ts

└─ analyzeBuild() → AnalysisResult (existing)

└─ for each failedTest:

 PredictionService.predict(failedTest, buildId, aggregateCtx)

 └─ 1. CacheLookup (predictions-cache.json)

 └─ HIT → return {source:'cache' | 'override'}

 └─ 2. OverrideLookup (kb-overrides.json, keyed same as cache)

 └─ HIT → return {source:'override'}

 └─ 3. RuleClassifier.classify(failedTest)

 └─ matched & confidence ≥ 0.85 → return {source:'rule'}

 └─ 4. KbMatcher.match(failedTest, aggregateCtx)

 └─ iterate all KB entries; compute weighted score

 └─ pick best; require score × entry.confidence ≥ 0.6

 └─ render reasonTemplate with Handlebars + failedTest vars

 → return {source:'kb', kbId, kbFilePath, suggestedFixes}

 └─ 5. No match

 → return {category:'Unknown', source:'unknown', confidence:0,

 reason:'No matching KB entry. Click "Suggest KB entry" to add one.'}

|



Response: AnalysisResult with `failedTests[i].prediction`

|

Client renders the Prediction column:

- category chip
- reason (truncated; " KB" link opens KbDrawer with bodyMarkdown)
- confidence bar
- Approve / Reject / Edit / + Suggest-KB buttons

...

9.2 R2 — Compare Builds across two environments (MD-KB)

...

User picks Env1 / Env2 (or pastes two build URLs)

|

Client: POST /api/env-compare

|

Server: envCompare.ts

```
├─ parallel analyzeBuild for both
├─ parallel PredictionService.predictBatch for both
├─ EnvCompareService.computeBuckets(env1, env2) (set math, same as primary)
└─ if (withNarrative):
    NarrativeRenderer.renderEnvDiff({ buckets, predictionsEnv1, predictionsEnv2 })
    ├─ group predictionsEnv1/2 by category → counts
    ├─ for each top-N cluster, pull fragment from kb/env-diff/<bucket>.md
    ├─ feed into Handlebars template `env-diff.hbs` (server-side)
    └─ output:
        {
          summary: "Env1 has 12 timeout failures absent in Env2 ...",
          env1RootCauses: [{category, count, note}], // note from KB fragments
          env2RootCauses: [...],
          keyDifferences: ["Only Env1 shows ADO-feed-5xx clusters", ...]
        }
```

|



Response: EnvCompareResult (same shape as primary variant)

...

`env-diff/\*.md` files hold reusable phrasings, e.g.:

```markdown

---

id: kb.env-diff.qa-only-cluster

appliesWhen:

bucket: onlyInEnv1

category: NetworkError

minCount: 3

---

{{count}} `{{category}}` failures appear **only** in {{env1.envName}}.

This typically indicates an environment-specific outage or throttling.

Inspect {{env1.envName}}'s ADO feed and Azure Storage account logs around  
{{firstFailure.startTime}}.

...

### 9.3 R3 — Dev PR correlation (MD-KB)

...

User pastes PR URL → Correlate

|

Client: POST /api/pr/correlate { buildUrl, prUrl }

|

Server: prCorrelate.ts

└─ provider auto-detect (ado | github) — same as primary

└─ provider.fetchPr + fetchChanges → PrRef + PrChange[]

└─ analyzeBuild(buildUrl) → failedTests[]

|

└─ for each failedTest:

PrCorrelationService.correlate(failedTest, changes):

└─ 1. RULE PASS (overlap + symbol + stack-frame) — same as primary

| produces { suspectHunks, matchedBy }

| if (no overlap) → emit {likelyCausedByPR:false, confidence:0,

| explanation:'No file/symbol overlap with PR.', matchedBy:[]}

|

└─ 2. KB PASS (only when suspectHunks non-empty):

enrich failedTest with suspectHunks (added/removed symbols, hunk text)

KbMatcher.match(enrichedTest, { scope: 'pr-related' })

• only consider entries under kb/pr-related/

• match keys may include `changedSymbolsAny`, `changeTypeAny`

→ render explanation via reasonTemplate

→ confidence = entry.confidence × overlapScore

→ likelyCausedByPR = confidence ≥ 0.6

|

Server writes pr-correlation-cache.json

|

Client renders verdict + suspect hunks + KB-rendered explanation

+ Approve / Reject / Edit / + Suggest-KB controls

...

Example `kb/pr-related/selector-rename.md`:

```markdown

id: kb.pr-related.selector-rename

category: PrRelated

confidence: 0.85

match:

errorMessageAny: ["NoSuchElementException", "element not found"]

```
changedSymbolsAny: ["By.Id", "By.CssSelector", "By.XPath"]
changeTypeAny: ["edit"]
reasonTemplate: |
  PR `{{pr.id}}` modifies a selector helper that the failing test relies on
  ({{matchedSymbols}}). This is the likely cause.
```

9.4 Feedback → KB PR loop

User clicks Approve / Reject / Edit / + Suggest-KB

|

Client: POST /api/feedback/prediction

|

Server: feedback.ts

```
|—— FeedbackStore.append(entry) → prediction-feedback.json (audit log)
|
|—— action === 'approve':
|   kb-overrides.json[key] = { kbId, frozen: true } // pins this KB entry
|
|—— action === 'reject':
|   kb-overrides.json[key] = { kbId: null, suppressedKbId: prev.kbId }
|   (next prediction will skip that KB entry for this key)
|
|—— action === 'edit':
|   kb-overrides.json[key] = { reason: edited, category: edited, source:'override' }
|   KbPrDrafter.queueDraft(failedTest, edited) → kb-pr-queue.json
|   (user sees "KB PR draft queued. Open PR?" toast)
|
|—— action === 'kb-pr':
    KbPrDrafter.openPr(draftId):
      1. Pick target repo from env (BBT_KB_REPO_URL).
      2. Build front-matter + body from queued draft.
      3. Commit to a new branch `kb/auto/<slug>-<timestamp>`.
      4. Open PR via AdoPrProvider or GitHubPrProvider (host detect).
      5. Add reviewers from kb/CODEOWNERS.
      6. Return { prUrl, prId }.
```

Once the PR is merged and the KB folder reloads (CI deploys the new `.md`, `chokidar` picks it up), the `kb-overrides.json` entry for that key can be safely removed (or kept as a permanent pin).

10. Rule Library (unchanged from primary variant)

`rule-patterns.json` still ships as a first-pass deterministic layer. Same shape, same six seed rules (`Timeout`, `ElementNotFound`, `AssertionMismatch`, `NetworkError`, `EnvironmentDown`, `Flaky`). The KB layer extends — not replaces — these.

11. Narrative Templates (replaces LLM prompts)

`NarrativeRenderer.ts` uses **Handlebars** with three template files under `dashboard/src/server/templates/`:

...

templates/

```
└─ env-diff.hbs          (R2 — narrative)
└─ pr-correlation.hbs    (R3 — explanation when no specific KB matched)
└─ prediction-fallback.hbs (R1 — used when KB matches but reasonTemplate is missing)
...

```

Example `env-diff.hbs`:

```
``handlebars
```

```

**{{env1.envName}} vs {{env2.envName}}** — {{buckets.failingInBoth.length}} failing in both,
{{buckets.onlyInEnv1.length}} only in {{env1.envName}}, {{buckets.onlyInEnv2.length}} only in {{env2.envName}}.

```

```

{{#each env1Clusters}}
- {{env1.envName}}-only cluster: {{count}}× **{{category}}** — {{note}}
{{/each}}

{{#each env2Clusters}}
- {{env2.envName}}-only cluster: {{count}}× **{{category}}** — {{note}}
{{/each}}

```

Key differences:

```
{#{each keyDifferences}}
- {#{this}}
{#{/each}}
...
```

Template helpers: `{{count}}`, `{{plural}}`, `{{shorten n}}`, `{{linkAdoBuild buildId}}`.

12. Security & Governance (delta)

| Area | Approach |

|-----|-----|

| ****No outbound AI traffic**** | The container makes ****no calls**** to AWS / OpenAI / Mistral. All prediction work is in-process. Easier network review. |

| ****ADO PAT / GitHub token**** | Unchanged from primary variant. GitHub token now also used to open KB PRs (when KB lives in GitHub). |

| ****KB repo write**** | `KbPrDrafter` uses a dedicated bot account / token (`BBT\_KB\_BOT\_TOKEN`) scoped only to PR creation in the KB repo. Never reused for build/test fetch. |

| ****PII / customer data**** | Failure messages never leave the server. The redactor in `PromptBuilder` is still applied before `kb-pr-queue.json` is written, so drafted PR bodies are PII-free. |

| ****Prompt injection**** | N/A — no LLM. Handlebars templates HTML-escape by default; rendered output is shown as plain text in React. |

| ****KB content trust**** | KB entries are code-reviewed via PR. `npm run kb:validate` runs in CI; PR is blocked on schema / regex / glob errors. `CODEOWNERS` on `kb/` enforces SDET-lead review. |

| ****Rate limiting**** | Still applied to `/api/predict/\*` and `/api/pr/correlate` to protect the build-fetch path (ADO REST is the rate-sensitive dependency now). |

13. Caching & Performance

| Layer | TTL / Invalidation |

|-----|-----|

| `predictions-cache.json` | Keyed by `sha256(testCaseTitle \| normalizedErrorSig \| buildDefinitionId \| kbVersion)`. `kbVersion` = git short-sha of the `kb/` folder; bumping the KB invalidates only relevant entries naturally on next miss. |

| `env-compare-cache.json` | Keyed by `sha256(buildId1 \| buildId2 \| narrativeFlag \| kbVersion)`. TTL = 7 days. |

| `pr-correlation-cache.json` | Keyed by `sha256(buildId \| prUrl \| prIterationId \| kbVersion)`. |

| `kb-overrides.json` | No TTL — explicit user action only. |

| In-memory | KB entries kept as a `Map<string, KbEntry>`; matcher iterates ~hundreds of entries in <5 ms (no embedding, no network). |

Typical end-to-end latency for `POST /api/predict/failures` on a build with 50 failures: **<150 ms** (dominated by ADO fetch, *not* prediction). Compare to ~30–90 s for the LLM variant when many cache misses occur.

14. Deployment & Config

No change to Azure Pipelines / Docker / Azure Storage. New env vars (LLM-related vars are dropped):

| Var | Purpose | Default |

|----|-----|-----|

| `BBT\_PAT\_FILE` | (existing) ADO PAT file path | `src/data/pat.json` |

| `BBT\_KB\_DIR` | Local path to the KB folder | `dashboard/kb` |

| `BBT\_KB\_REPO\_URL` | Target repo for KB PRs (ADO or GitHub URL) | (required for `kb-pr` action) |

| `BBT\_KB\_BOT\_TOKEN` | Token for the bot account that opens KB PRs | (required for `kb-pr` action) |

| `BBT\_KB\_RELOAD\_ON\_CHANGE` | Watch the KB folder (set `false` in immutable container) | `true` |

| `BBT\_KB\_MATCH\_THRESHOLD` | Minimum `score × confidence` to accept a KB match | `0.6` |

| `GITHUB\_TOKEN` | (existing) GitHub PR fetch (R3) | optional |

When the KB lives in the **same repo** as the dashboard, no separate repo URL is needed and `KbPrDrafter` falls back to the existing ADO PR API with the standard PAT (subject to ADO branch policies).

15. Output / Export Formats

Identical to primary variant. Export columns simply read from `prediction.source = 'kb'` and include `kbId` / `kbFilePath` instead of `modelId`. Adds one extra column in the `.xlsx` export: **KB Link** (hyperlink to the `.md` file in the repo browser).

16. Feedback, Observability & Quality Metrics

- **`prediction-feedback.json`** — same append-only audit log; gains a `kbPrDraftId` field.

- **`/api/feedback/stats`** returns extra:

- `kbCoverage` = `(predictions where source ∈ {'rule','kb','override'}) / total predictions`

- `kbPrsOpened`, `kbPrsMerged` (queries the KB repo via the bot token)

- `unknownTop10` — the ten most-frequent `unknown` predictions, surfaced as a **KB backlog** in the Quality page so SDETs know what to author next.

- Server logs every prediction with `{kbId, score, matchedKeys, latencyMs}`. Zero per-call cost replaces the old token/\$ logging.

17. Risks & Mitigations

| Risk | Mitigation |

|-----|-----|

| KB coverage gaps → many `Unknown` predictions | `unknownTop10` widget + 1-click "Suggest KB entry" → tight authoring loop; track `kbCoverage` as a release KPI. |

| KB drift (stale entries match the wrong failures) | `lastReviewed` front-matter; CI warns on entries unreviewed >180 days; conflict-checker in `kb:validate`. |

| Regex / glob authoring errors | `ajv` JSON-schema validation + `kb:validate` CI gate + entry preview endpoint (`POST /api/kb/validate`). |

| Two entries match the same failure | Scoring + `lastReviewed` tiebreak; `kb:validate` flags pairs whose match specs overlap by ≥80 %. |

| Hot-reload races on busy server | `chokidar` debounce 500 ms; matcher reads a snapshot Map; swap is atomic. |

| Bot token compromise (KB repo write) | Scope token to KB repo only; rotate quarterly; PR review required to merge. |

| Loss of LLM narrative richness | Acceptable per design; template narrative + KB body Markdown shown side-by-side gives equivalent practical value for SDET triage. |

18. Phased Delivery

| Phase | Scope | Exit criteria |

|-----|-----|-----|

| **P1** | RuleClassifier + Prediction column (no KB yet) | ≥60 % of failures categorized. |

| **P2** | `kb/` folder, `KbLoader`, `KbMatcher`, 20 seed entries | Combined rule+KB coverage ≥85 % on a one-week sample. |

| **P3** | Environment registry + Compare Builds two-env mode + `NarrativeRenderer` | QA-vs-Stage comparison demo with rendered narrative. |

| **P4** | PR correlation (rule pass + `kb/pr-related/` matcher) | Top-3 PR-related KB entries authored; demo on a real PR. |

| **P5** | Feedback (Approve/Reject/Edit) + `kb-overrides.json` | Edits override predictions immediately. |

| **P6** | `KbPrDrafter` + `POST /api/kb/pr` + bot account | Edit action can open a real KB PR end-to-end. |

| **P7** | Excel/CSV/JSON exports for all three views + Quality page (incl. `unknownTop10`) | UAT sign-off. |

19. Comparison: LLM Variant vs MD-KB Variant

| Dimension | LLM variant | **MD-KB variant** |

|---|---|---|

| Cloud AI dependency | AWS Bedrock | **None** |

| Per-call cost | \$0.003–\$0.015 per failure | **\$0** |

| End-to-end latency (50 failures) | 30–90 s | **<150 ms** |

| Handles novel failures | Yes — graceful generalization | **No** — falls back to `Unknown` until KB entry is authored |

| Explainability | Confidence + RAG exemplar ids | **Direct link to the `.md` file that fired** |

| Auditability | Prediction-feedback log + RAG store | **Full git history of every prediction rule** |

| Long-term improvement | RAG memory grows automatically | **KB grows via human-reviewed PRs (slower but higher signal)** |

| Compliance footprint | Cross-cloud egress review | **In-process only — easiest review** |

| Best fit when... | Failure variety is high and KB authoring can't keep up | **Failure variety is bounded and SDETs want full control** |

A future hybrid is possible: **try KB first, then LLM** (toggle via env var `BBT\_PREDICT\_LLM\_FALLBACK=true`). That migration requires zero schema changes — `FailurePrediction.source` already includes both `kb` and `llm` enumerations conceptually; only the orchestrator step needs to be added.

20. Out of Scope (v1)

- Auto-generation of KB entries from LLM (would defeat the purpose of this variant).
- Multi-tenant KB (single shared KB assumed).
- KB entry semantic search beyond simple text search (acceptable at $\leq 1\,000$ entries; revisit with embeddings later).
- Multi-replica scale-out — single-replica Docker assumed for v1; KB reload uses local `chokidar`.

21. Appendix — Endpoint Quick Reference (MD-KB variant)

```

### # Prediction

POST /api/predict/failures

POST /api/predict/failure

GET /api/predict/cache/:buildId

DELETE /api/predict/cache/:buildId

### # Environment Compare

GET /api/environments

POST /api/env-compare

GET /api/env-compare/cache/:key

### # PR Correlation

POST /api/pr/resolve

POST /api/pr/correlate

GET /api/pr/correlate/cache/:key

### # Feedback

POST /api/feedback/prediction (action: approve | reject | edit | kb-pr)

GET /api/feedback/stats

### # KB management (NEW — variant-only)

GET /api/kb

GET /api/kb/:id

POST /api/kb/reload

POST /api/kb/validate

POST /api/kb/suggest

POST /api/kb/pr

### # Existing (unchanged)

POST /api/analyze (extended: ?withPrediction=true)

GET /api/health

GET /api/pat/status

POST /api/pat/reload

```

End of MD-KB-variant architecture document.